

Exercise 1

Write an API to list all the `Band` objects in the system with the corresponding `Musician` data nested within each `Band` object.

To build a band listing API, you need schemas for the `Band` and `Musician` objects. These can both be subclasses of `ModelSchema` to minimize the amount of code required. The following listing shows the required code.

Listing E.47 Serializers for the Band and Musician objects

```
# RiffMates/bands/api.py
...
from ninja import ModelSchema
...
from bands.models import Band, Musician
...
class MusicianOut(ModelSchema):
    class Meta:
        model = Musician
        fields = ["id", "first_name", "last_name", "birth",
                  "description",]

class BandSchema(ModelSchema):
    musicians: list[MusicianOut]

    class Meta:
        model = Band
        fields = ["id", "name"]
```

Subclass `ModelSchema` to quickly create a serializer for the `Musician` class

The inner `Meta` class gets used to define the serializer's association with the ORM class.

The fields value contains the list of fields from `Musician` that participate in this serializer.

The model field defines which ORM model is associated with this schema.

To nest musicians in a band serializer, define a field that relates to the `Musician` schema.

With the appropriate serializers in place, the API endpoint is a simple view that returns a list of all the `Band` objects. The following listing shows the code.

Listing E.48 A migration script with a data operation in it

```
# RiffMates/bands/api.py
...
router = Router()
...
@router.get("/bands/", response=list[BandSchema])
def list_bands(request):
    return Band.objects.all()
```

API endpoints are registered against the single `Router` defined in the file, created earlier in chapter 11.

A band-listing endpoint responds with a list of `BandSchema` serializations.

You return a `QuerySet` of `Band` objects to `Ninja`, and it takes care of the rest.

Exercise 2

Write an API to update the fields of a `Musician` object, requiring authentication to do so.